Available online at www.sciencedirect.com

SciVerse ScienceDirect

Solar Energy 86 (2012) 1323–1337

SOLAR ENERGY

www.elsevier.com/locate/solener

Five new algorithms for the computation of sun position from 2010 to 2110

Roberto Grena *

C.R. ENEA Casaccia, via Anguillarese 301, 00123 Roma, Italy

Received 25 November 2011; received in revised form 20 January 2012; accepted 22 January 2012
Available online 28 February 2012

Communicated by: Associate Editor L. Vant-Hull

Abstract

Five algorithms for sun position computation, with validity from 2010 to 2110, are proposed and discussed. The algorithms have different accuracy levels, with maximum errors in the solar position spanning from 0.19° to 0.0027°, thus covering a wide range of possible applications. The algorithms are optimized in order to reduce their computational cost as much as possible. The extended time range of validity allows users to employ these algorithms even in long-term projects.

Keywords: Sun position; Right ascension; Declination; Hour angle; Zenith; Azimuth

1. Introduction

Accurate computation of solar position plays a fundamental role in solar energy applications, especially for concentrating systems. The required accuracy varies over a wide range, depending on the application: flat systems tolerate errors of a few degrees without significant losses, while high-concentration systems can require an accuracy of the order of 0.01°. More specific applications, such as the calibration of pyranometers (Reda and Andreas, 2004), require an even greater accuracy.

Despite its apparent simplicity, accurate computation of the solar position is a quite difficult task. Indeed, the apparent motion of the sun is subject to a high number of perturbations: precession and nutation of the rotation axis of the earth, perturbations caused by the moon, the decreasing rotation speed of the earth, and the effects of other planets. All these factors affect the computation in different ways.

Several algorithms for computing the solar position with different levels of accuracy and complexity can be found in the solar engineering literature. Simple formulas (Cooper, 1969; Spencer, 1971; Swift, 1976; Lamm, 1981) that find the declination or the equation of time usually have errors of the order of tenths of degree. A more complex algorithm was proposed by Pitman and Vant-Hull (1978), with a maximum error of 0.02°; some years later Walraven published another algorithm (Walraven, 1978) with an error of 0.013°, followed by corrections and comments (Walraven, 1979; Archer, 1980; Wilkinson, 1981; Ilyas, 1983; Pascoe, 1984). Another algorithm was proposed by Michalsky 10 years later (Michalsky, 1988); it is based on The Astronomical Almanac (1985), and it reduces the maximum error to 0.01°. Two other algorithms with lower errors, but also with a shorter time range of validity, were proposed in the following years (Blanco-Muriel et al., 2001; Grena, 2008); the first was developed at the Plataforma Solar de Almeria (it will be indicated as PSA), the second at ENEA (it will be indicated as ENEA). All these algorithms have quite simple implementations and low computational complexity.

* Tel.: +39 0630486856.
E-mail address: roberto.grena@enea.it

0038-092X/$ - see front matter © 2012 Elsevier Ltd. All rights reserved.
doi:10.1016/j.solener.2012.01.024

A more accurate algorithm found in the literature is an astronomical algorithm far more complex than the algorithms listed above (Meeus, 1998) that Reda and Andreas brought to the attention of the solar engineering community (Reda and Andreas, 2004). It is indicated as Solar Position Algorithm (SPA). This algorithm has a maximum error of 0.0003° on the period from 2000 BC to 6000 AD. Such an error is extremely small from the point of view of solar engineering, but not for astronomical applications (e.g., stellar parallaxes are orders of magnitude smaller).

The other algorithms listed above, specifically chosen for solar engineering applications, are far less accurate, and they usually have a much shorter time range validity. For instance, the three most recent methods (Michalsky, PSA, ENEA) declare maximum errors of 0.01°, 0.008° and 0.0027°, respectively, and their validity time ranges are 1950–2050, 1995–2015, 2003–2023. These algorithms are far simpler than SPA, with a computational complexity that is estimated as tens of times lower (see Section 2 for a short discussion of computational complexity). This feature can have a crucial importance if low cost, error-proof control systems are to be devised. However, even if the accuracy of these three algorithms is sufficient for most solar engineering applications, the limit on the validity range can pose some problems. In particular, two among these algorithms are not suitable to be employed in new long-term projects, and the Michalsky algorithm is also well beyond its half-life.

The previous discussion is a motivation to present in this paper five new algorithms with a validity range of 100 years, starting from 2010. Different levels of accuracy (and complexity) are considered, from a very simple algorithm with 0.19° of maximum error, suitable for flat and low-concentrating systems, to a quite complex algorithm with 0.0027° of maximum error. All these algorithms are devised to reduce computational complexity as much as possible, according to the criteria given in Section 2. Their complexity remains well below the complexity of the SPA method, and it is comparable to the complexity of solar engineering-specific algorithms (as the examples cited above). All the five algorithms give both global and local coordinates: they calculate the right ascension α and the declination δ, the hour angle H, the zenith z and the azimuth Γ. The algorithms are presented also in a short version, which finds only right ascension, declination and hour angle: such results are often sufficient for many applications.

The aim of the present work is to give a set of algorithms suitable for a wide range of applications, able to give complete information on the sun position with a validity time range that reaches well beyond the expected life for a solar plant planned in the next few years. This last feature is especially important, since modifying the computation of the solar position during the life of a solar plant can be troublesome, in particular if the control system is formed by stand-alone, small closed devices, or if the plant is sold to a third party.

The paper is organized as follows. Section 2 lists the required input and output data and it includes a short discussion on computational complexity. In Section 3, the five algorithms are given as step-by-step procedures. Section 4 is devoted to error analysis and to comparison with existing algorithms.

2. Input, output, and complexity

2.1. Input data

In order to compute the sun position, the following data must be provided:

1. *Time.* Year (y), month (m), day (d) and universal hour (h) must be given. The universal hour is defined as the standard Universal Time (UT) at Greenwich meridian (also indicated as UT1), and it can be computed (within an error of 0.9 s) as the local standard time minus the time zone correction with respect to Greenwich time. For instance, 13h00′ in Rome corresponds to $h = 12.0$. The universal hour must be given in decimal format, converting minutes and seconds to decimal fractions of hours (e.g., 12h30′ becomes $h = 12.5$). UT is based on the earth rotation; since the rotation is not perfectly regular, this time scale is not regular either, since it is kept aligned with the earth rotation. All the algorithms presented here immediately convert the input time data to the number of days measured from the midpoint of the interval of validity (the beginning of the year 2060): this time scale will be called t in the following.
2. *Difference between Terrestrial Time, TT and Universal Time, UT.* Astronomical calculations need a regular time scale independent of the terrestrial rotation; one of these time scales is indicated as Terrestrial Time (TT). The difference $\Delta\tau = \mathrm{TT} - \mathrm{UT}$ expresses the irregularity of the earth rotation, that is, its nonuniform deceleration. The value of $\Delta\tau$ is experimentally determined and it can be found in tables. Fig. 1, built with

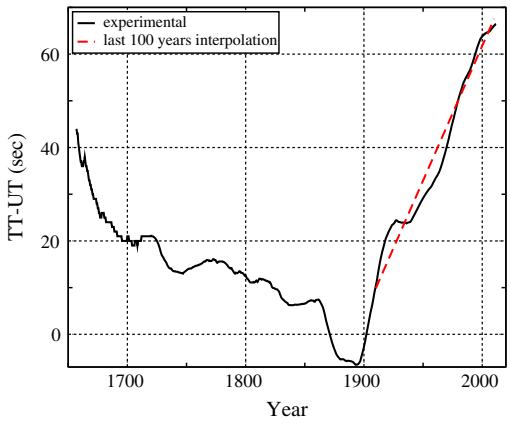

Fig. 1. Variation of $\Delta\tau = \mathrm{TT} - \mathrm{UT}$ from 1650 to now; data taken from http://maia.usno.navy.mil/. The linear extrapolation given in Eq. (1) is drawn for the last 100 years.

data taken from http://maia.usno.navy.mil/, shows the value of $\Delta\tau$ in the past. A linear extrapolation done on the last 100 years is

$$\Delta\tau = 96.4 + 0.00158t \text{ s}, \quad (1)$$

where the parameter t is defined as above, and $\Delta\tau$ is given in seconds (even if t is measured in days). This extrapolation cannot be very accurate, since $\Delta\tau$ changes irregularly. However, an error up to 30 s in $\Delta\tau$ does not significantly affect the calculations in any of the algorithms, as it is shown below. If Algorithm 5 is excluded, then errors up to 100 s are acceptable.

3. *Position.* Geographical longitude θ and latitude φ must be given. The most widely used reference system is the standard geodetic system WGS84, used for example in the GPS positioning system. If this reference system is adopted, the computed local coordinates z and Γ will be expressed with respect to the standard vertical and south direction given by WGS84. However, it is not necessary to use WGS84; one can choose instead the best local system, or a system adopting the gravitational vertical. This arbitrariness is due to the fact that the coordinates are not used to identify a position on the earth surface, but only to determine the orientation of the local system of polar coordinates[1]; so any reasonable coordinate system can be employed. Of course, once a reference system has been chosen, it must be used consistently, without switching to other systems.
4. *Pressure P and temperature T.* These data are required for the computation of the refraction correction. This is a separate issue not strictly related to the sun position algorithms, but since it can have a significant effect when the sun is low, a formula for the correction is given at the end.

If the short version of the algorithm is used (see the next subsection for a detailed description), the required input parameters are only y, m, h, $\Delta\tau$ and θ. However, φ is then presumably required for the conversion of the data to useful quantities, as the angle of incidence on a surface.

In this paper, errors on the results are generally expressed in degrees to improve readability. In the algorithms, however, all the input and output data are in radians, since this is the unit usually required by calculators. In particular, all the coefficients given in the formulas of Section 3 are correct only for radians, and the results are all given in radians.

The accuracy required for the input data can be estimated from the accuracy of the algorithm. A separate discussion is needed for the computation of global and local coordinates. For global coordinates (right ascension α and declination δ), only the time data are used; since the angular velocity of the sun in the celestial reference sphere is roughly 10^{-5} deg/s, an error of some tens of seconds in the input data does not significantly affect the accuracy of any of the presented algorithms, if only global coordinates are concerned. For the same reason, the difference $\Delta\tau$ (which is only used to compute global coordinates) does not need to be very accurate. An error of 30 s is perfectly acceptable for all the algorithms, since it corresponds to an angular error of about 0.0003°, only 1/3 of the standard deviation of the most accurate algorithm. For the first four algorithms, errors up to 100 s do not cause problems. One may therefore use extrapolations on $\Delta\tau$ over all the relevant time range without loss of accuracy. In fact, $\Delta\tau$ can be eliminated from the input data and set at the extrapolated value (1), or even set at the current value, and it is unlikely that this would cause problems to the algorithms. However, it should be remembered that $\Delta\tau$ is an unpredictable quantity, and no guarantees can be given on the accuracy of extrapolated values. In the following, $\Delta\tau$ is left as an input parameter for generality.

A much greater accuracy on input data is required when computing local sun coordinates. Indeed, an error of Δa on the angular position on the earth (θ and φ) induces an error of the order of Δa on the vertical and south orientation, and consequently on the sun position. For this reason, if E is the error of the algorithm on the sun position, the errors on the longitude and latitude must be significantly lower than E, otherwise the final error will be dominated by the error in the input data and the accuracy of the algorithm will be spoiled. For the most accurate algorithms, this limit can be very strict. For example, an error of 0.0027° on the latitude (equal to the maximum error of Algorithm 5) corresponds to a distance of 300 m on the earth surface. A large solar field could require different input coordinates for a few different positions in the field, if the full accuracy of the algorithm is required. Of course, algorithms can be used with less accurate input parameters, but the final error on the local coordinates of the sun will be mainly induced by the error on the input data, rather than by the algorithmic error.

A similar remark holds for time error, with the exception of $\Delta\tau$ (which is not required for local coordinates). Since the angular velocity of rotation of the earth is roughly 0.004 deg/s, the error (in seconds) on time accuracy should be significantly lower than $E/0.004$ (with E in degrees). In the case $E = 0.0027°$, this bound is about 0.7 s.

A final remark should be made on the temperature and pressure data required for the refraction correction. With the formula used here (24), the correction is directly proportional to P and inversely proportional to $273 + T$; so, the relative error on the correction will be equal to the relative error on P, or on $273 + T$. In absolute terms, for an elevation of 5° the correction is roughly 0.15° (1 atm, 20 °C); this means that a change of 10% in P or $273 + T$ causes an error of 0.015°. The error becomes rapidly negligible as the sun elevation increases. As a rule of thumb, an

[1] In fact, there is an exception, and it is the computation of the parallax correction to the zenith (see below), that depends on the position on the earth; however, the accuracy required on the position is low, and all the systems are equivalent for this purpose.

error within a few % on $273 + T$ or P is acceptable for all the algorithms, for sun elevations >5°.

Units used for the input data in all the following formulas are: hours for h; seconds for $\Delta\tau$; radians for θ and φ; atmospheres for P; Celsius degrees for T.

2.2. Output data

The five computed quantities are the two standard global coordinates (right ascension α and declination δ), and three local coordinates (the hour angle H, the zenith z and the azimuth Γ). The global coordinates α and δ are coordinates in a system with the polar axis coincident with the earth rotation axis, and the origin of the azimuthal angle in the zodiacal ascending point (the position of the sun at the spring equinox). The right ascension is the azimuthal angle (like the terrestrial longitude) measured from the ascending point, and the declination is the angle with respect to the equatorial plane (like the terrestrial latitude). The global coordinates depends only on the time.

Local coordinates z (or, equivalently, the elevation $e = \pi/2 - z$) and Γ are referred to a local polar system with the polar axis along the vertical. The zenith z is the angle between the vertical axis and the sun position, and the azimuth Γ is the azimuthal angle on the horizontal plane, with $\Gamma = 0$ towards south, and positive direction towards west. Γ spans from $-\pi$ rad to $+\pi$ rad (negative towards the east hemisphere),[2] z from 0 rad to π rad ($z = 0$ if the sun is exactly above the observer). As already pointed out, the definition of the vertical is given by the coordinate system used (usually WGS84): this means that the inclination of the vertical with respect to the earth equator plane is φ.

Beside the two polar local coordinates, another frequently used coordinate, the hour angle H, is given in output. The hour angle is the azimuthal angle of the sun with the polar axis aligned with the earth axis, and $H = 0$ when the sun is to the south (the true solar mid-day).

Note that, in principle, the explicit calculation of the local coordinates z and Γ is not required. The incidence angle on a surface, for example, can be computed from the couple (δ, H), without passing through (z, Γ). Indeed, (δ, H) is in fact an earth-fixed, polar reference frame, and the incidence angle on a surface can be found expressing the normal to the surface in this frame and computing the scalar product between the normal and the sun position. It will be clear from the structure of the algorithms that this property can save a significant amount of computational time, since H is computed before the other local coordinates and with fewer trigonometric functions. For this reason, for each algorithm two versions are considered: a short algorithm that outputs only α, δ and H, and the complete algorithms, with the final computation of z and Γ and the refraction correction.

However, two problems are encountered when using the short versions of the algorithms. The first is the impossibility of introducing the refraction correction: the calculation of the refraction requires the sun elevation (or equivalently the zenith). So, the short version can be used only when refraction is negligible.

The second problem is that the computed coordinates δ and H are *geocentric*, i.e., the polar system in which they are defined has the origin in the centre of the earth; but in the context of solar applications it is more useful to consider a *topocentric* system, with the origin on the earth surface. The ratio (Earth radius/Sun distance) gives the order of magnitude of the angular error (in radians) obtained when using geocentric coordinates instead of the true local topocentric coordinates. This ratio corresponds roughly to 0.0025°, and the effect cannot be ignored in the most accurate algorithms. The correction can be introduced in a simple and effective way only after the elevation (or zenith) has been computed, and it takes the form of a simple parallax correction to the elevation; this correction is not present in the short version of the algorithms.[3] So, short versions and long versions of the algorithms are not rigorously equivalent from the point of view of accuracy. This fact can be neglected for the first two algorithms, since the correction is far smaller than the algorithms' output errors, but it becomes quite significant for Algorithms 3 and 4 (the maximum error increases of 25% using the short version), and it has remarkable consequences on Algorithm 5 (the maximum error is almost doubled when using the short version). In fact, short versions are especially recommended for the first two algorithms, where they allow for a significant decrease in computational time without spoiling accuracy. For the other algorithms, long versions should be preferred, since the difference in computational cost is relatively lower, the accuracy difference becomes significant, and the refraction correction can be introduced.

In this paper, the accuracies given for the algorithms will always be that of the long versions: only for Algorithms 1 and 2 the given accuracies also hold for short versions.

All the algorithms give the output data in radians, for all the angles. For azimuthal-type angles (α, H, and Γ) a range must be specified. The chosen range is $[0, 2\pi]$ for α, and $[-\pi, \pi]$ for H and Γ. These ranges are purely conventional; H and Γ are chosen to start from $-\pi$ in order not to have an abrupt change of 2π at midday, when solar plants are working (the 2π change happens at solar midnight).

The two angles α and H are often given in "angular hours", with an hour corresponding to 15°. The conversion can be simply made by multiplying the output value in radians by $12/\pi$; H can then be shifted forward by 12 h in order to have the value 12 (instead of 0) at midday, if

[2] Sometimes, different starting points, orientation and ranges are used for the azimuth; so, it is advisable to always check which convention is being used.

[3] An alternative strategy consists in the introduction of the apparent topocentric quantities α_t, δ_t and H_t (see for example Reda and Andreas, 2004; Grena, 2008); however, the computation of these quantities adds complications, while the correction of the zenith is extremely simple and straightforward, as it will be seen later.

desired. Fractions of hours can be converted in "minutes" and "seconds" (with the caution of not confusing these units with the true angular minutes and seconds of degree, which are 15 times smaller).

Table 1 summarizes the input and output data, with the symbols used throughout the paper, the units of measurement adopted in the algorithms, the range of values required in input or given in output, and the error limit for the input data (for computation of local coordinates).

2.3. Complexity

Since all the operations required by the algorithms must be performed by control systems, the computational complexity of the algorithms is an especially important issue. A reduced complexity allows the algorithms to be employed on simple and cheap control systems, even for single elements of the solar field, thus reducing the cost and the error risk.

In numerical analysis, the complexity of an algorithm traditionally used to be measured by counting the required number of products and divisions (Isaacson and Keller, 1966), and ignoring sums and subtractions (considered as negligible with respect to multiplications and divisions). All the more complex functions and operations should then be reduced to elementary operations. However, this point of view poses two problems. First, it is now obsolete: in modern computers, products are fast as sums, while divisions are significantly slower than both. Secondly, the decomposition of complex functions in elementary operations is not easy and it requires knowledge of the routines used by the compiler.

Here, a more experimental approach is used. An experimental test on calculation time, made on a Sony Vaio laptop equipped with an Intel Core-Duo P8700 processor with 2.53 GHz clock, using the GNU C++ compiler, gives the results shown in Table 2, for the most common operations and functions. One can see that sums and products are almost undistinguishable from the point of view of computational time; divisions and square roots are roughly 10 times slower; trigonometric functions, direct and inverse, have a computational time among 25 and 50 times a sum. Truncation to integer type (with a C++ cast) is faster than sums, and conversion of integer to double precision type is a bit slower but comparable; the same holds for the absolute value function. Elevating to a non-integer power is the most expensive function analyzed (almost 100 times a sum). The time required for a conditional choice (if statement) is very low (0.6 times a sum). However, these proportions can change using other machines, programming languages, or compilers.

Table 1
Input and output quantities of the algorithms, with the required or given units of measurement, and the acceptable error for the input data. E denotes the desired accuracy of the output.

Quantity	Symbol	Unit	Range	Max. error
Input data:				
Year	y		2010–2110	
Month	m		1–12	
Day	d		1–31	
UT	h	h	0–24	$\ll E/14.4$
TT-UT	Δt	s		30
Longitude	θ	rad	$[0, 2\pi]$	$\ll E$
Latitude	φ	rad	$[-\pi/2, \pi/2]$	$\ll E$
Pressure	P	atm		0.05 (elev. >5 deg)
Temperature	T	°C		10 (elev. >5 deg)
Output data:				
Right ascension	α	rad	$[0, 2\pi]$	
Declination	δ	rad	$[-\pi/2, \pi/2]$	
Hour angle	H	rad	$[-\pi, \pi]$	
Zenith	z	rad	$[0, \pi]$	
Azimuth	Γ	rad	$[-\pi, \pi]$	

Table 2
Time required for different operations on a laptop with an Intel Core-Duo P8700 processor, 2.53 GHz clock, using the GNU C++ compiler. The time required for a sum is considered as 1.

Operation	Time
$+$	1.00
$\times$	1.00
$/$	10.5
$\sqrt{}$	10.8
Sin	28.5
Cos	29.6
Tan	40.4
Arcsin	49.3
Arccos	49.5
Arctan	26.4
If statement	0.59
Cast to int	0.53
Cast to double	1.46
Absolute value	1.74
Real power	94.3
Exponential	42.2
Logarithm	68.4

Based on the results listed in the table, estimates on the computational cost of other similar operations can be derived. According to this analysis, the modulo function (the remainder of the division) has a computational weight slightly above the weight of a division.[4] The function arctan2(x, y), present in many programming languages, would have roughly the same weight of a simple arctan. Here $\xi = \text{arctan2}(x, y)$ is the arctangent function with the "correct" quadrant assignment: it requires x and y to be proportional to sine and cosine, respectively, of the output angle ξ, and it returns ξ in the range $[-\pi, \pi]$.[5]

In this paper, the aforementioned results are used to calculate the complexity of the algorithms, by identifying a class of operations that are assumed to roughly have the same computational weight of a sum (subtractions, products, conversion of types, absolute values, conditional

[4] Here the modulo function is assumed to be applied to real numbers. Modulo between integer numbers can be faster.

[5] In some programming languages or software packages, the order of the arguments or the range of the result can be different: the user might want to check such details before using the function.

choices), a second class that includes operations whose computational weight is 10 times a sum (divisions, square roots, moduli), and the class of trigonometric functions, which are considered as having complexity 30 times a sum. No other functions are used in the algorithms.

These considerations point to a few ways to reduce computational complexity. The number of trigonometric functions should be reduced as much as possible (in fact, in all the algorithms in circulation they account for a large fraction of the computational cost). However, due to the intrinsic periodicity of the sun motion, it is not possible to avoid trigonometric functions altogether. Some tricks can be employed, though, to reduce the number of trigonometric functions. If the sine and cosine of an angle x are computed, all the harmonics relative to that angle (e.g., $\sin(2x)$, $\cos(2x)$, etc.) can be found using only products and divisions; so, higher harmonics can be computed with a small additional cost. If the sine of an angle is known, together with the sign of the cosine, then the cosine can be obtained from the formula $\pm\sqrt{1-(\sin x)^2}$, and vice versa, since square roots are roughly three times faster than trigonometric functions[6]; this trick can be employed when calculating trigonometric functions of declination, latitude, earth axis inclination and sun elevation, which always have a positive cosine. These trivial tricks can reduce the computational cost of the algorithms. However, six final computations of trigonometric functions are required for the geometrical conversion to the local coordinates z and Γ. For simple algorithms, these computations form the largest part of the computational burden (in the first two algorithms, 6 of the 9 required trigonometric functions are used for this conversion). For this reason, the short version of the algorithms, which gives (α, δ, H), is recommended especially for the simplest algorithms (in the first two algorithms, only two trigonometric functions are required in the short versions).

Caution should be used also in making divisions and modulo operations, which are 10 times slower than products. On the other hand, truncations, absolute values, roundings and type conversions do not seem to be a heavy computational burden.

These considerations can lead to somewhat counterintuitive expressions. A complex polynomial expression can result computationally less expensive than a single trigonometric function. Long chains of products and sums to calculate the higher harmonics of a trigonometric function (e.g., $\sin(4x)$) can be at first glance less readable than the simple writing of $\sin(4x)$. This also explains the absence of the ecliptic longitude from the first two algorithms: the ecliptic longitude is of no interest to solar engineering, but it is usually computed as a first step in many algorithms (e.g. Michalsky, 1988; Blanco-Muriel et al., 2001; Grena, 2008) since it allows to compute α and δ with compact expressions. However, these expressions involves trigonometrical functions; for this reason, this step is skipped in the first two algorithms and both α and δ are computed directly with less compact, but more computationally efficient expressions. However, starting from Algorithm 3, accuracy concerns require the computation of the ecliptic longitude as a first step.

If a central control system is used, with a modern and fast PC to compute the sun position, the computational burden becomes a negligible aspect, while the readability and error-proofness of the algorithms becomes more important. For this reason, each algorithm is described twice, first in the most readable and intuitive way (directly implementable if the computational burden is not a problem), and then in the step-by-step version optimized for reduction of the computational cost.

3. Algorithms

The description of the algorithms is organized as follows. Common initial and final computations are given once in Sections 3.1 and 3.7; the initial computations build the time scale used in all the algorithms, the final computations (avoidable if the short algorithms are used) are geometrical transformations that allow to obtain z and Γ, with the parallax and refraction corrections. Between these two subsections, the central body of each algorithm is given in Sections 3.2–3.6, one for each algorithm; the point at which the computation can be stopped in the short algorithm is clearly indicated.

The construction of the algorithms relies on physical considerations that allow to find (approximately) the fundamental frequencies and the coefficients of the formulas. This is followed by a numerical fine-tuning for the time range 2010–2110, using SPA data. For Algorithm 5, a Fourier analysis was also made to extract initial approximate values for the additional frequencies that appear in the algorithm (some of which do not have a clear physical interpretation, as it will be seen).

The errors given for the algorithms, in absence of other indications, are intended as absolute angular errors on the local position (see Section 4).

3.1. Starting point: time scale computation

In all the algorithms, the first step is the time computation. The time t used here is the number of days starting from the beginning of the year 2060 (roughly the midpoint of the interval 2010–2110), according to UT. The initial steps are simply the conversion of the date and hours in this time scale. First of all, if $m \leqslant 2$, the month must be increased by 12 and the year must be decreased by 1 (the

[6] This trick is not useful when the sign of the cosine is not known, since identifying the correct sign requires a modulo and the procedure is overall as complex as calculating the cosine. Moreover, this trick should be avoided when *relative* errors are important, since when the square root goes to 0 the relative error on the computed trigonometric function can become significant. This is not the case for the algorithms described here, where only absolute errors are important.

"corrected" month and year number will be called $\tilde{m}$ and $\tilde{y}$). Then, the time t is obtained with the formula

$$t = \mathrm{INT}(365.25(\tilde{y} - 2000)) + \mathrm{INT}(30.6001(\tilde{m} + 1)) - \mathrm{INT}(0.01\tilde{y}) + d + h/24 - 21958; \quad (2)$$

In this formula, the function INT indicates rounding towards zero: i.e., $\mathrm{INT}(3.7) = 3$, $\mathrm{INT}(-4.2) = -4$. In C or C++, a simple cast to integer can be used (but then a re-conversion to a double should be made). If the time range considered in the computations does not span beyond the year 2099, the third term (the last INT function) can be directly replaced with 20. The division $h/24$ can be written as a product ($0.0416667h$) to save computational effort.

Beside the time t, dependent on the earth rotation (it is based on the Universal Time, UT), another time t_e based on TT and independent of the earth rotation is computed, adding to t the difference $\Delta\tau$ (converted in days):

$$t_e = t + 1.1574 \times 10^{-5}\Delta\tau. \quad (3)$$

The time t_e will be used to establish the global sun position, of course independent from the earth rotation, while the time t is required to compute the local coordinates.

This is the initial step for all the algorithms given in this paper. Note that the computational cost is very low, since all the operations performed belongs to the class of operation whose computational cost is comparable to the cost of sums and products. The step-by-step procedure to perform the computation is

1. if $m \leqslant 2$, $\tilde{m} = m + 12$ and $\tilde{y} = y - 1$, otherwise $\tilde{m} = m$ and $\tilde{y} = y$;
2. calculate $t = \mathrm{INT}(365.25(\tilde{y} - 2000)) + \mathrm{INT}(30.6001(\tilde{m} + 1)) - \mathrm{INT}(0.01\tilde{y}) + d + 0.0416667h - 21958$;
3. calculate $t_e = t + 1.1574 \times 10^{-5}\Delta\tau$.

3.2. Algorithm 1

This is a very simple algorithm that allows the computation of the sun position within a maximum angular error of 0.19° (with a standard deviation of 0.14°). The short version only requires two trigonometric functions. Right ascension and declination are computed using a fundamental angular frequency $\omega = 0.017202786\ \mathrm{day}^{-1}$ up to the second harmonic. The formulas (all in radians) are:

$$\alpha = -1.38880 + 1.72027920 \times 10^{-2}t_e + 3.199 \times 10^{-2}\sin(\omega t_e) - 2.65 \times 10^{-3}\cos(\omega t_e) + 4.050 \times 10^{-2}\sin(2\omega t_e) + 1.525 \times 10^{-2}\cos(2\omega t_e); \quad (4)$$

$$\delta = 6.57 \times 10^{-3} + 7.347 \times 10^{-2}\sin(\omega t_e) - 3.9919 \times 10^{-1}\cos(\omega t_e) + 7.3 \times 10^{-4}\sin(2\omega t_e) - 6.60 \times 10^{-3}\cos(2\omega t_e); \quad (5)$$

$$H = 1.75283 + 6.3003881t + \theta - \alpha. \quad (6)$$

The values of α and H must then be replaced by the corresponding angles in their conventional range: $\alpha \to \mathrm{mod}(\alpha, 2\pi)$, and $H \to \mathrm{mod}(H + \pi, 2\pi) - \pi$.

The short algorithm ends here. The long algorithm requires the additional steps described in Section 3.7.

The computationally optimized procedure for this calculation is:

1. calculate $s_1 = \sin(\omega t_e)$, and $c_1 = \cos(\omega t_e)$;
2. calculate $s_2 = 2s_1c_1$, and $c_2 = (c_1 + s_1)(c_1 - s_1)$;
3. calculate α:

$$\alpha = -1.38880 + 1.72027920 \times 10^{-2}t_e + 3.199 \times 10^{-2}s_1 - 2.65 \times 10^{-3}c_1 + 4.050 \times 10^{-2}s_2 + 1.525 \times 10^{-2}c_2;$$

4. shift α to its conventional range: $\alpha \to \mathrm{mod}(\alpha, 2\pi)$;
5. calculate δ:

$$\delta = 6.57 \times 10^{-3} + 7.347 \times 10^{-2}s_1 - 3.9919 \times 10^{-1}c_1 + 7.3 \times 10^{-4}s_2 - 6.60 \times 10^{-3}c_2;$$

6. calculate $H = 1.75283 + 6.3003881t + \theta - \alpha$;
7. shift H to its conventional range: $H \to \mathrm{mod}(H + \pi, 2\pi) - \pi$.

3.3. Algorithm 2

This can be considered as a simple brute-force improvement on Algorithm 1; α and δ are computed up to the fourth harmonic of the same fundamental angular frequency $\omega = 0.017202786\ \mathrm{day}^{-1}$. This allows to reduce the maximum error to 0.034° (standard deviation 0.013°) without adding any "heavy" functions, since higher harmonics can be obtained by sums and products. However, the expressions become quite cumbersome. The formulas (all in radians) are:

$$\alpha = -1.38880 + 1.72027920 \times 10^{-2}t_e + 3.199 \times 10^{-2}\sin(\omega t_e) - 2.65 \times 10^{-3}\cos(\omega t_e) + 4.050 \times 10^{-2}\sin(2\omega t_e) + 1.525 \times 10^{-2}\cos(2\omega t_e) + 1.33 \times 10^{-3}\sin(3\omega t_e) + 3.8 \times 10^{-4}\cos(3\omega t_e) + 7.3 \times 10^{-4}\sin(4\omega t_e) + 6.2 \times 10^{-4}\cos(4\omega t_e); \quad (7)$$

$$\delta = 6.57 \times 10^{-3} + 7.347 \times 10^{-2}\sin(\omega t_e) - 3.9919 \times 10^{-1}\cos(\omega t_e) + 7.3 \times 10^{-4}\sin(2\omega t_e) - 6.60 \times 10^{-3}\cos(2\omega t_e) + 1.50 \times 10^{-3}\sin(3\omega t_e) - 2.58 \times 10^{-3}\cos(3\omega t_e) + 6 \times 10^{-5}\sin(4\omega t_e) - 1.3 \times 10^{-4}\cos(4\omega t_e); \quad (8)$$

$$H = 1.75283 + 6.3003881t + \theta - \alpha. \quad (9)$$

The values of α and H must be replaced by the corresponding angles in their conventional range: $\alpha \to \mathrm{mod}(\alpha, 2\pi)$, and $H \to \mathrm{mod}(H + \pi, 2\pi) - \pi$.

The short algorithm ends here. The long algorithm requires the additional steps described in Section 3.7.

The computationally optimized procedure for this calculation is:

1. calculate $s_1 = \sin(\omega t_e)$, and $c_1 = \cos(\omega\ t_e)$;
2. calculate $s_2 = 2s_1c_1$, and $c_2 = (c_1 + s_1)(c_1 - s_1)$;
3. calculate $s_3 = s_2c_1 + c_2s_1$, and $c_3 = c_2c_1 - s_2s_1$;
4. calculate $s_4 = 2s_2c_2$, and $c_4 = (c_2 + s_2)(c_2 - s_2)$;
5. calculate α:

$$\alpha = -1.38880 + 1.72027920 \times 10^{-2} t_e + 3.199 \times 10^{-2} s_1 - 2.65 \times 10^{-3} c_1 + 4.050 \times 10^{-2} s_2 + 1.525 \times 10^{-2} c_2 + 1.33 \times 10^{-3} s_3 + 3.8 \times 10^{-4} c_3 + 7.3 \times 10^{-4} s_4 + 6.2 \times 10^{-4} c_4;$$

6. shift α to its conventional range: $\alpha \to \mathrm{mod}(\alpha, 2\pi)$;
7. calculate δ:

$$\delta = 6.57 \times 10^{-3} + 7.347 \times 10^{-2} s_1 - 3.9919 \times 10^{-1} c_1 + 7.3 \times 10^{-4} s_2 - 6.60 \times 10^{-3} c_2 + 1.50 \times 10^{-3} s_3 - 2.58 \times 10^{-3} c_3 + 6 \times 10^{-5} s_4 - 1.3 \times 10^{-4} c_4;$$

8. calculate $H = 1.75283 + 6.3003881t + \theta - \alpha$;
9. shift H to its conventional range: $H \to \mathrm{mod}(H + \pi, 2\pi) - \pi$.

3.4. Algorithm 3

A simple harmonic analysis of α and δ does not allow to improve accuracy further (adding harmonics above the 4th order does not significantly improve accuracy). For this reason, in order to obtain a maximum error below 0.01°, the sun ecliptic longitude must be computed and then converted to the system of coordinates (α, δ). The resulting algorithm has more compact formulas, but it is computationally heavier with respect to the previous two, due to the trigonometric transformations required to switch from the ecliptic longitude to the coordinates α and δ. The maximum error is reduced to 0.0094° (standard deviation 0.0033°).

The apparent ecliptic longitude λ of the sun from the earth can be computed (in radians) with good accuracy considering only the fundamental frequency $\omega_a = 0.0172019715\ \mathrm{day}^{-1}$ up to the second harmonics:

$$\lambda = -1.388803 + 1.720279216 \times 10^{-2} t_e + 3.3366 \times 10^{-2} \times \sin(\omega_a t_e - 0.06172) + 3.53 \times 10^{-4} \sin(2\omega_a t_e - 0.1163). \quad (10)$$

The earth axis inclination is then required. It is given, within an error of 0.003°, by the formula

$$\epsilon = 4.089567 \times 10^{-1} - 6.19 \times 10^{-9} t_e. \quad (11)$$

From these two quantities, α and δ are obtained as

$$\alpha = \arctan 2(\sin\lambda \cos\epsilon, \cos\lambda); \quad (12)$$

$$\delta = \arcsin(\sin\lambda \sin\epsilon). \quad (13)$$

The function $\mathrm{arctan2}(x, y)$ is present in many programming languages; if the function is not present, it can be easily built by calculating the usual arctangent of the ratio of the two parameters, and then adding π if the parameter proportional to the cosine is negative, shifting the result of -2π if it exceeds π.[7]

The hour angle H is then computed:

$$H = 1.7528311 + 6.300388099t + \theta - \alpha. \quad (14)$$

The value of H must be replaced by the corresponding angle in its conventional range: $H \to \mathrm{mod}(H + \pi, 2\pi) - \pi$. α, being the result of a arctan2 function, is in the range $[-\pi, \pi]$, and it must be shifted forward by 2π if it is negative to obtain the correct conventional range.

The short algorithm ends here. The long algorithm requires the additional steps described in Section 3.7.

The computationally optimized procedure for this calculation is:

1. calculate λ:

$$\lambda = -1.388803 + 1.720279216 \times 10^{-2} t_e + 3.3366 \times 10^{-2} \times \sin(\omega_a t_e - 0.06172) + 3.53 \times 10^{-4} \sin(2\omega_a t_e - 0.1163).$$

2. calculate ϵ:

$$\epsilon = 4.089567 \times 10^{-1} - 6.19 \times 10^{-9} t_e;$$

3. calculate $s_\lambda = \sin\lambda$, and $c_\lambda = \cos\lambda$;
4. calculate $s_\epsilon = \sin\epsilon$, and $c_\epsilon = \sqrt{1 - s_\epsilon^2}$;
5. calculate $\alpha = \mathrm{arctan2}(s_\lambda c_\epsilon, c_\lambda)$;
6. if $\alpha < 0$, $\alpha \to \alpha + 2\pi$;
7. calculate $\delta = \arcsin(s_\lambda s_\epsilon)$;
8. calculate $H = 1.7528311 + 6.300388099t + \theta - \alpha$;
9. shift H to its conventional range: $H \to \mathrm{mod}(H + \pi, 2\pi) - \pi$.

3.5. Algorithm 4

This fourth algorithm is very similar to the previous one, but with the nutation correction inserted. This correction reduces of about 30% the maximum error on global coordinates, and of 35% the standard deviation. However, the correction does not affect very much the errors on local coordinates (the standard deviation is reduced of 10%, and the maximum error is almost unchanged). The reason of this difference will be discussed in the next section. It is unlikely that this algorithm will be really useful for solar engineering, where only local coordinates are used; how-

[7] This approach may prove useful even if the function arctan2 is available, since the implementation of arctan2 of the compiler can be computationally heavier than the implementation of arctan.

ever, the algorithm is described anyway for completeness, and because the correction is required to bring the global maximum error below 0.01°.

This time, the starting quantity is the heliocentric ecliptic longitude of the Earth, computed as above up to the second harmonics (in radians), with the same fundamental frequency $\omega_a = 0.0172019715\ \text{day}^{-1}$:

$$L = 1.752790 + 1.720279216\times10^{-2}t_e + 3.3366\times10^{-2} \times \sin\times(\omega_a t_e - 0.06172) + 3.53\times10^{-4}\sin(2\omega_a t_e - 0.1163). \quad (15)$$

The nutation acts with a frequency $\omega_n = 9.282\times10^{-4}$ day^{-1}. The angle $v = \omega_n t_e - 0.8$ is computed, and the correction to the apparent ecliptic longitude is given by $\Delta\lambda = 8.34\times10^{-5}\sin v$. The apparent geocentric ecliptic longitude of the sun is given by

$$\lambda = L + \pi + \Delta\lambda \quad (16)$$

(the π factor is due to the change of reference system, from heliocentric to geocentric).

The same angle is used to compute the correction to the earth axis inclination, now given by

$$\epsilon = 4.089567\times10^{-1} - 6.19\times10^{-9}t_e + 4.46\times10^{-5}\cos v. \quad (17)$$

From these quantities, α and δ are computed as in the previous algorithm:

$$\alpha = \arctan 2(\sin\lambda\cos\epsilon, \cos\lambda);$$
$$\delta = \arcsin(\sin\lambda\sin\epsilon). \quad (18)$$

The hour angle H is then computed, with the nutation correction:

$$H = 1.7528311 + 6.300388099t + \theta - \alpha + 0.92\Delta\lambda. \quad (19)$$

The value of H must be replaced by the corresponding angle in its conventional range: $H \to \text{mod}(H+\pi, 2\pi) - \pi$. α, being the result of a arctan2 function, is in the range $[-\pi, \pi]$; when it is negative, it must be shifted forward by 2π to obtain the correct conventional range.

The short algorithm ends here. The long algorithm requires the additional steps described in Section 3.7.

The computationally optimized procedure for this calculation is:

1. calculate L:
   $$L = 1.752790 + 1.720279216\times10^{-2}t_e + 3.3366\times10^{-2} \times \sin(\omega_a t_e - 0.06172) + 3.53\times10^{-4}\sin(2\omega_a t_e - 0.1163).$$
2. calculate $v = \omega_n t_e - 0.8$;
3. calculate $\Delta\lambda = 8.34\times10^{-5}\sin v$;
4. calculate $\lambda = L + \pi + \Delta\lambda$;
5. calculate ϵ:
   $$\epsilon = 4.089567\times10^{-1} - 6.19\times10^{-9}t_e + 4.46\times10^{-5}\cos v.$$
6. calculate $s_\lambda = \sin\lambda$, and $c_\lambda = \cos\lambda$;
7. calculate $s_\epsilon = \sin\epsilon$, and $c_\epsilon = \sqrt{1 - s_\epsilon^2}$;
8. calculate $\alpha = \arctan 2(s_\lambda c_\epsilon, c_\lambda)$;
9. if $\alpha < 0$, $\alpha \to \alpha + 2\pi$;
10. calculate $\delta = \arcsin(s_\lambda s_\epsilon)$;
11. calculate H:
    $$H = 1.7528311 + 6.300388099t + \theta - \alpha + 0.92\Delta\lambda;$$
12. shift H to its conventional range: $H \to \text{mod}(H+\pi, 2\pi) - \pi$.

3.6. Algorithm 5

This final algorithm is quite different from the previous ones, since many perturbations must be taken into account in order to decrease the maximum error well below 0.01°. For this reason, the complexity is increased, and beside some corrections of clear physical interpretation (e.g., the effect of the Moon, Venus and Jupiter) there are other harmonic corrections with a less clear meaning, due to more complex actions (such as the combined influence of other distant planets). The most computationally intensive part of the algorithm is found especially in the initial calculation of L.

In the computation of L, the same fundamental frequency $\omega_a = 0.0172019715\ \text{day}^{-1}$ is considered, with a third harmonic added; moreover, two symmetric frequencies $\omega_a \pm \beta$, very close to the fundamental frequency, are included. Both corrections at $\omega_a \pm \beta$ depends, with good approximation, only on the cosine, and have coefficients with opposite sign. For this reason they can be transformed to a non-linear correction involving a single new trigonometric function (since $\cos(\omega_a t_e + \beta t_e) - \cos(\omega_a t_e - \beta t_e) = -2\sin(\omega_a t_e)\ \sin(\beta t_e)$). Beside these, seven new frequencies ω_i, $i = 1 \ldots 7$, are included.

Including all these corrections, the heliocentric ecliptic longitude is given by

$$L = L_0 + L_1 t_e + \sum_{k=1}^{3}(a_k \sin(k\omega_a t_e) + b_k \cos(k\omega_a t_e)) + d_\beta \sin(\omega_a t_e)\sin(\beta t_e) + \sum_{i=1}^{7} d_i \sin(\omega_i t_e + \phi_i). \quad (20)$$

In Table 3, numerical values are given for all the coefficients and frequencies in the formula. The last frequency ω_7 corresponds to a period around 29.5 days, and it clearly represents the effect of the moon. Two other easily interpretable frequencies are ω_3, corresponding to 584 days, and ω_4, corresponding to 399 days: these are, respectively, the periods between two conjunctions of Venus and Jupiter with the Earth.

The calculation then proceeds as in the Algorithm 4, for the computation of the nutation correction, λ, ϵ, α, δ and H: the nutation correction depends on the same frequency $\omega_n = 9.282\times10^{-4}\ \text{day}^{-1}$.

Table 3
Values of the parameters used in the computation of L, in Algorithm 5.

L_0	1.7527901				
L_1	$1.7202792159 \times 10^{-2}$				
ω_a	0.0172019715	a_1	3.33024×10^{-2}	b_1	-2.0582×10^{-3}
		a_2	3.512×10^{-4}	b_2	-4.07×10^{-5}
		a_3	5.2×10^{-6}	b_3	-9×10^{-7}
β	2.92×10^{-5}	d_β	-8.23×10^{-5}		
ω_1	1.49×10^{-3}	d_1	1.27×10^{-5}	ϕ_1	−2.337
ω_2	4.31×10^{-3}	d_2	1.21×10^{-5}	ϕ_2	3.065
ω_3	1.076×10^{-2}	d_3	2.33×10^{-5}	ϕ_3	−1.533
ω_4	1.575×10^{-2}	d_4	3.49×10^{-5}	ϕ_4	−2.358
ω_5	2.152×10^{-2}	d_5	2.67×10^{-5}	ϕ_5	0.074
ω_6	3.152×10^{-2}	d_6	1.28×10^{-5}	ϕ_6	1.547
ω_7	2.1277×10^{-1}	d_7	3.14×10^{-5}	ϕ_7	−0.488

$$v = \omega_n t_e - 0.8;$$
$$\Delta\lambda = 8.34 \times 10^{-5} \sin v;$$
$$\lambda = L + \pi + \Delta\lambda;$$
$$\epsilon = 4.089567 \times 10^{-1} - 6.19 \times 10^{-9} t_e + 4.46 \times 10^{-5} \cos v;$$
$$\alpha = \arctan 2(\sin\lambda \cos\epsilon, \cos\lambda);$$
$$\delta = \arcsin(\sin\lambda \sin\epsilon);$$
$$H = 1.7528311 + 6.300388099t + \theta - \alpha + 0.92\Delta\lambda.$$

The value of H must be replaced by the corresponding angle in its conventional range: $H \to \mathrm{mod}(H + \pi, 2\pi) - \pi$. α, being the result of a arctan2 function, is in the range $[-\pi, \pi]$, and it must be shifted forward by 2π if it is negative to obtain the correct conventional range.

The short algorithm (not recommended: the error is strongly increased) ends here. The long algorithm requires the additional steps described in the next subsection.

The computationally optimized procedure for this calculation is

1. calculate $s_1 = \sin(\omega_a t_e)$, and $c_1 = \cos(\omega_a t_e)$;
2. calculate $s_2 = 2s_1c_1$, and $c_2 = (c_1 + s_1)(c_1 - s_1)$;
3. calculate $s_3 = s_2c_1 + c_2s_1$, and $c_3 = c_2c_1 - s_2s_1$;
4. calculate L (using the coefficients and frequencies of Table 3):
$$L = L_0 + L_1 t_e + \sum_{k=1}^{3} (a_k s_k + b_k c_k) + d_\beta s_1 \sin(\beta t_e) + \sum_{i=1}^{7} d_i \sin(\omega_i t_e + \phi_i).$$
5. calculate $v = \omega_n t_e - 0.8$;
6. calculate $\Delta\lambda = 8.34 \times 10^{-5} \sin v$;
7. calculate $\lambda = L + \pi + \Delta\lambda$;
8. calculate ϵ:
$$\epsilon = 4.089567 \times 10^{-1} - 6.19 \times 10^{-9} t_e + 4.46 \times 10^{-5} \times \cos v.$$
9. calculate $s_\lambda = \sin\lambda$, and $c_\lambda = \cos\lambda$;
10. calculate $s_\epsilon = \sin\epsilon$, and $c_\epsilon = \sqrt{1 - s_\epsilon^2}$;
11. calculate $\alpha = \arctan 2(s_\lambda c_\epsilon, c_\lambda)$;
12. if $\alpha < 0$, $\alpha \to \alpha + 2\pi$;
13. calculate $\delta = \arcsin(s_\lambda s_\epsilon)$;
14. calculate H:
$$H = 1.7528311 + 6.300388099t + \theta - \alpha + 0.92\Delta\lambda;$$
15. shift H to its conventional range: $H \to \mathrm{mod}(H + \pi, 2\pi) - \pi$.

3.7. Final steps

The final steps, to be used only in the long versions, are common to all the algorithms. They calculate z, Γ, the parallax correction to z (i.e. the shift from geocentric to topocentric angular coordinates) and the refraction correction to z.

The elevation angle is the complementary angle of the zenith ($e = \pi/2 - z$). The elevation e_0 (uncorrected for the refraction and the parallax) is obtained with the formula

$$e_0 = \arcsin(\sin\varphi \sin\delta + \cos\varphi \cos\delta \cos H). \quad (21)$$

The parallax correction to e_0, that shifts the elevation in order to obtain the topocentric elevation, is given by

$$\Delta_p e = -4.26 \times 10^{-5} \cos e_0. \quad (22)$$

Since $\cos e_0$ is always positive, and $\sin e_0$ is already known (it is the argument of arcsin in (21)), the parallax correction can be computed without additional trigonometric functions. The elevation e_p corrected for the parallax (but not for the refraction) is simply $e_p = e_0 + \Delta_p e$.

The azimuth Γ is computed with

$$\Gamma = \arctan 2(\sin H, \cos H \sin\varphi - \tan\delta \cos\varphi). \quad (23)$$

The final step is the computation of the refraction correction. It is independent of the main body of the algorithm, and the formula presented here (taken from Reda and Andreas (2004)) can be replaced with any other formulas that better describes local conditions or specific meteorological situations. The formula given here depends on e_p, P and T:

$$\Delta_r e = \frac{0.08422P}{273 + T} \frac{1}{\tan\left(e_p + \frac{0.003138}{e_p + 0.08919}\right)}. \quad (24)$$

Of course, this formula is meaningful only when $e_p + \Delta_r e$ is positive.

From the elevation, the zenith (with all the corrections) is immediately obtained:

$$z = \frac{\pi}{2} - e_p - \Delta_r e. \quad (25)$$

This ends the calculation.

Note that only the zenith is corrected for the difference between topocentric and geocentric coordinates; the effects on Γ are negligible.

The computationally optimized procedure of the final steps for all the algorithms is:

Table 4
Errors, time range and computational cost of the algorithms, compared with three algorithms from the literature.

	Error		Time validity	Computational cost	
	σ	Range		Short	Long
Algorithm 1:					
α	0.07	[−0.16 0.16]	2010–2110	134	428
δ	0.12	[−0.18 0.19]			
H	0.07	[−0.16 0.16]			
z	0.11	[−0.19 0.19]			
Γ	0.17				
Δa_g	0.14	[0 0.19]			
Δa_l	0.14	[0 0.19]			
Algorithm 2:					
α	0.013	[−0.041 0.034]	2010–2110	161	455
δ	0.005	[−0.015 0.019]			
H	0.013	[−0.038 0.038]			
z	0.007	[−0.034 0.034]			
Γ	0.019				
Δa_g	0.013	[0 0.038]			
Δa_l	0.013	[0 0.034]			
Algorithm 3:					
α	0.0044	[−0.0134 0.0116]	2010–2110	278	572
δ	0.0018	[−0.0050 0.0046]			
H	0.0031	[−0.0113 0.0119]			
z	0.0021	[−0.0093 0.0092]			
Γ	0.0043				
Δa_g	0.0046	[0 0.0125]			
Δa_l	0.0033	[0 0.0094]			
Algorithm 4:					
α	0.0029	[−0.0098 0.0089]	2010–2110	347	641
δ	0.0009	[−0.0037 0.0032]			
H	0.0030	[−0.0117 0.0122]			
z	0.0015	[−0.0091 0.0093]			
Γ	0.0038				
Δa_g	0.0029	[0 0.0091]			
Δa_l	0.0029	[0 0.0094]			
Algorithm 5:					
α	0.0008	[−0.0022 0.0022]	2010–2110	635	929
δ	0.0003	[−0.0011 0.0009]			
H	0.0013	[−0.0050 0.0050]			
z	0.0005	[−0.0025 0.0027]			
Γ	0.0012				
Δa_g	0.0008	[0 0.0023]			
Δa_l	0.0009	[0 0.0027]			
Michalsky:					
α	0.0046	[−0.0125 0.0138]	1950–2050		530
δ	0.0019	[−0.0046 0.0055]			
H	0.0032	[−0.0124 0.0115]			
z	0.0031	[−0.0128 0.0098]			
Γ	0.0049				
Δa_g	0.0047	[0 0.0139]			
Δa_l	0.0041	[0 0.0128]			
PSA:					
α	0.0039	[−0.0095 0.0108]	1995–2015		589
δ	0.0012	[−0.0033 0.0043]			
H	0.0028	[−0.0101 0.0091]			
z	0.0016	[−0.0074 0.0074]			
Γ	0.0037				
Δa_g	0.0039	[0 0.0111]			
Δa_l	0.0027	[0 0.0074]			

(*continued on next page*)

Table 4. (*continued*)

	Error		Time validity	Computational cost	
	σ	Range		Short	Long
ENEA:					
α	0.0007	[−0.0021 0.0022]	2003–2023		846
δ	0.0003	[−0.0007 0.0009]			
H	0.0007	[−0.0026 0.0020]			
z	0.0006	[−0.0029 0.0028]			
Γ	0.0016				
Δa_g	0.0007	[0 0.0021]			
Δa_l	0.0009	[0 0.0029]			
SPA:			2000 BC–6000 AD		>10,000

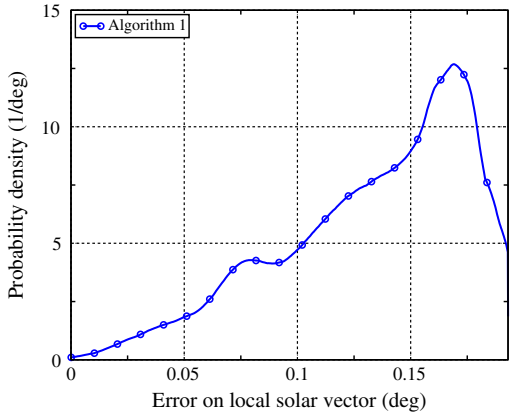

Fig. 2. Distribution of the errors on local solar vector for Algorithm 1.

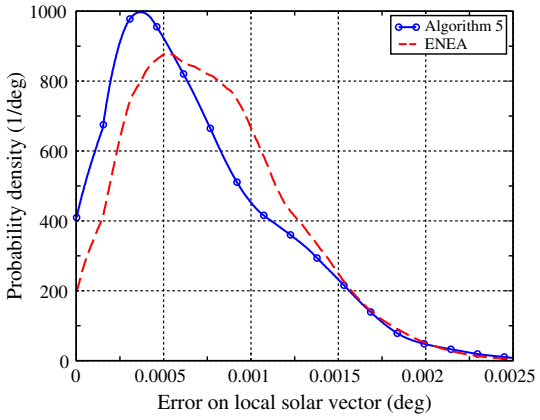

Fig. 4. Distribution of the errors on local solar vector for Algorithm 5.

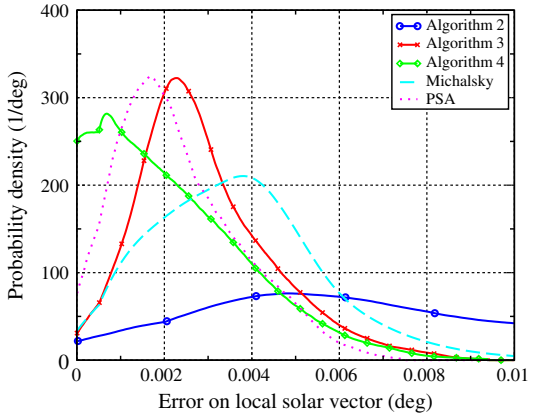

Fig. 3. Distributions of the errors on local solar vector for Algorithms 2–4.

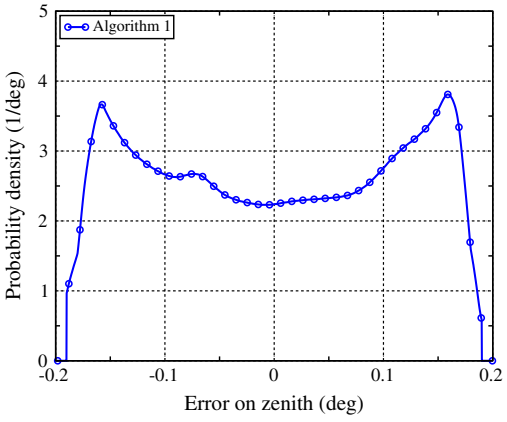

Fig. 5. Distribution of the errors on zenith for Algorithm 1.

1. calculate $s_\varphi = \sin\varphi$, and $c_\varphi = \sqrt{1 - s_\varphi^2}$;
2. calculate $s_\delta = \sin\delta$, and $c_\delta = \sqrt{1 - s_\delta^2}$;
3. calculate $s_H = \sin H$, and $c_H = \cos H$;
4. calculate $s_{e0} = s_\varphi s_\delta + c_\varphi c_\delta c_H$:
5. calculate $e_p = \arcsin s_{e0} - 4.26 \times 10^{-5}\sqrt{1 - s_{e0}^2}$;
6. calculate Γ:

$$\Gamma = \arctan 2(s_H, c_H s_\varphi - s_\delta c_\varphi / c_\delta);$$

7. calculate the refraction correction to the elevation:

$$\Delta_r e = \frac{0.08422P}{273 + T}\frac{1}{\tan\left(e_p + \frac{0.003138}{e_p + 0.08919}\right)};$$

8. calculate the Zenith:

$$z = \frac{\pi}{2} - e_p - \Delta_r e.$$

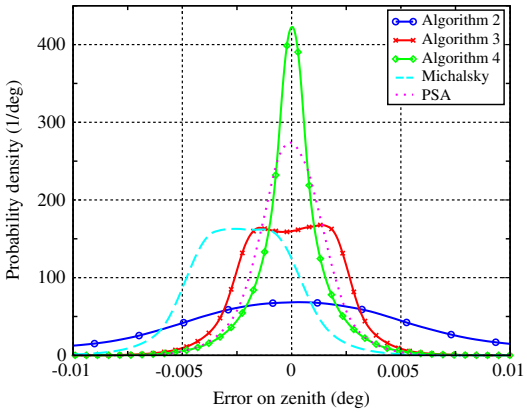

Fig. 6. Distributions of the errors on zenith for Algorithms 2–4.

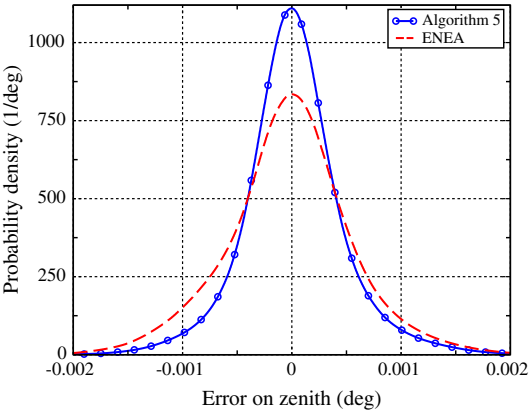

Fig. 7. Distribution of the errors on zenith for Algorithm 5.

4. Errors and computational cost

Two types of errors are considered: the standard deviation, that is, the error one can reasonably expect to have in a measure, and the maximum error, that is, the upper limit of the error. The standard deviation is the most statistically interesting quantity, while the maximum error is useful when certified results are needed to ensure the correct functioning of a tracking system.

The errors are computed by comparing the results of the algorithms to the results obtained from the SPA algorithm. Indeed, the error of the SPA algorithm is far smaller than the errors of all the algorithms presented here; therefore, the results obtained via SPA can be chosen as reference. The errors are computed by randomly choosing a date in the range 2010–2110, an hour and a location on the earth surface, by computing the errors on all the interesting quantities, and by repeating the procedure and collecting data until the distribution of errors is built. The random choice of time and locations allows to avoid systematic effects or underestimation of the errors due to a specific location or time sequence. 20 million trials were made to build the distributions shown below.

The quantites considered in the error computation are the global coordinates α and δ, and the local coordinates H, z and Γ. The hour angle H is compared to the topocentric H_t found by SPA, since this is the local quantity used for systems on the earth surface. However, the most important quantity is the absolute angular error, which is defined as the angle between the computed solar vector and the "real" one (here "real" means "obtained from SPA"). The absolute angular error on the global position is given by

$$\Delta a_g = \sqrt{(\Delta\alpha\cos\delta)^2 + \Delta\delta^2}, \tag{26}$$

and the absolute angular error on the local position is given by

$$\Delta a_l = \sqrt{(\Delta\Gamma\sin z)^2 + \Delta z^2}. \tag{27}$$

For a solar energy application, Δa_l is the most interesting quantity and it is used to assess the accuracy of the algorithm. All the errors declared for the algorithms must be intended as absolute angular errors on the local position, unless otherwise noted.

Special care should be taken when discussing the azimuth maximum error. Since Γ is undefined when the sun is exactly at the zenith, its error can become very large when $z \to 0$. This has nothing to do with the real accuracy of the position, since Δa_l remains small (the error on Γ is multiplied by $\sin z$, which goes to 0). So, if the location is not fixed, the maximum error on Γ has no meaning, and it only depends on how close to the zenith the sun gets in all the random choices of time and locations. For this reason, no error range for Γ is computed. Note that the same problem does not affect the other two azimuthal angles (α and H), since the corresponding "zenith-like" angles ($\pi/2 - \delta$ in both cases) are never smaller than 1.15 rad in absolute value.

In the error calculation, refraction effects are excluded, in order to compare all the algorithms on the same basis (e.g., PSA does not compute refraction effects), and to exclude effects that are only due to the approximation of the refraction effect. Moreover, errors caused by refraction are more likely due to changes in the physical conditions and to the validity of the approximations, than to algorithmic errors. Formula (24) is in fact totally independent of the algorithms, and it can be replaced with other formulas or omitted.

Table 4 shows all the errors computed in this way. The table shows the standard deviation for each quantity, and the range of errors obtained. The maximum error is the largest, in absolute value, between the two extremes of the range. The errors for the three most recent simple algorithms (Michalsky, PSA and ENEA) are also shown; they are computed as explained above, choosing times in the respective time range of validity of the algorithms. For this reason, these errors can be slightly larger than the bounds declared by the authors, since the tests

presented in the original papers were usually made in less general conditions (the Michalsky algorithm was tested over a year, PSA and ENEA in a fixed location). However, differences are small and do not affect the applicability of the algorithms.

In the same table, the computational cost of the algorithms is shown. The computational cost is computed according to the method described in Section 2: sums, products, type conversions, absolute values, if statements count as 1; square roots, divisions and moduli count as 10; trigonometric functions of all kinds count as 30. Refraction corrections, when present, are included in the count. The computational cost for PSA, Michalksy and ENEA algorithms is computed applying to the algorithms the same optimization described in the paper for the reduction of trigonometric functions (the three algorithms as described in the original papers have larger computational costs).

The computational cost of the five presented algorithms remains well below the cost of the SPA algorithm, and it is of the same order as the algorithms specifically aimed at solar engineering.

One can see that Algorithm 1 has a maximum error lower than 0.2°, with a very small computational cost (at least in the short version), and it is suitable for flat or low concentrating systems. However, with a small increase in computational effort (Algorithm 2), the error can be reduced to less than 0.04°, a sufficient accuracy for many applications (e.g., linear troughs). With a different approach and an extra amount of computation, the maximum error can be brought below 0.01° (Algorithms 3 and 4). Algorithm 4 appears quite useless for solar engineering, since the only significant error reduction it introduces with respect to the Algorithm 3 are on the global error, but it is interesting because of the introduction of the nutation correction. Nutation acts significantly on global coordinates, while a partial cancellation of the correction makes the nutation effect less important for local coordinates. Indeed, the change in α and the direct correction of the hour angle H in formula (19) act with opposite signs on H. This also explains why, in Algorithm 3, the local error is significantly lower than the global error: errors due to the neglected nutation affect the global coordinates more then the local ones.

Algorithm 5 has a larger complexity when compared to the other four, but it remains in the complexity class of engineering-specific algorithms. With a maximum error of 0.0027°, it is the most accurate algorithm (except SPA) proposed until now for solar engineering, with a validity of 100 years (the ENEA algorithm, with a similar accuracy, only works for 20 years).

Figs. 2–4 show the probability distribution of Δa_l; three figures are used in order to have homogeneity of scale in each figure. Fig. 1 shows the distribution for Algorithm 1, where a strong systematic effect is present (this explains why the maximum of the distribution is close to the maximum value). Fig. 2 shows the distribution of errors for Algorithms 2–4, compared with Michalksy and PSA. Fig. 4 shows the error distributions for Algorithm 5 and ENEA. Figs. 5–7 show error distributions on the zenith. The systematic effect for Algorithm 1 (Fig. 5) is evident from the shape of the distribution. In Fig. 6, one can see the effect of the parallax correction by observing that the error distribution of the Michalsky algorithm (the only algorithm without this correction) is not centered in 0, but it has a bias of 0.002°.

5. Conclusions

A set of new, simple algorithms for computing the solar position is proposed, for a period (2010–2110) which is not covered by any other accurate algorithm proposed for solar engineering except the very complex SPA. The algorithms have different accuracies and cover all the possible applications in solar engineering; at the same time, their complexity is kept to a minimum, and a computationally optimized version is given for each of them. One of these algorithms (Algorithm 5) is very accurate, with a maximum error of only 0.0027°. The long validity time range assures the applicability of the algorithms to any system or plant that can be planned now or in the near future, for all its life cycle. The algorithms can also be used in simulations requiring a large number of accurate sun position calculations, at the same time avoiding to add a heavy computational burden.

A C++ header file containing the five algorithms can be downloaded from http://www.solaritaly.enea.it/StrSunPosition/SunPositionEn.php.

Acknowledgements

Many thanks to Paola Boito (Université de Limoges, France) who read the draft of the paper, contributing with corrections and suggestions. Thanks to Francesco Spinelli and the administrators of the site www.solaritaly.enea.it who accepted to host the code on the site and put it online.

References

Archer, C.B., 1980. Comments on calculating the position of the Sun. Solar Energy 25, 91.

Blanco-Muriel, M., Alarcon-Padilla, D.C., Lopea-Moratalla, T., Lara-Coira, M., 2001. Computing the solar vector. Solar Energy 70, 431–441.

Cooper, P.I., 1969. The absorption of radiation in solar stills. Solar Energy 12, 333–346.

Grena, R., 2008. An algorithm for the computation of the solar position. Solar Energy 82, 462–470.

Ilyas, M., 1983. Solar position programs: refraction, sidereal time and Sunset/Sunrise. Solar Energy 31, 437–438.

Isaacson, E., Keller, H.B., 1966. Analysis of Numerical Methods. John Wiley and Sons.

Lamm, L.O., 1981. A new analytic expression for the equation of time. Solar Energy 26, 465.

Meeus, J., 1998. Astronomical Algorithms, second ed. Willmann-Bell Inc.

Michalsky, J.J., 1988. The astronomical Almanac's algorithm for approximate solar position (1950–2050). Solar Energy 40, 227–235.

Pascoe, D.J.B., 1984. Comments on Solar position programs: refraction, sidereal time and Sunset/Sunrise. Solar Energy 34, 205–206.

Pitman, C.L., Vant-Hull, L.L., 1978. Errors in locating the Sun and their effect on solar intensity predictions. In: Meeting of the American Section of the International Solar Energy Society, Denver, 28 August 1978, pp. 701–706.

Reda, I., Andreas, A., 2004. Solar position algorithm for solar radiation applications. Solar Energy 76, 577–589.

Spencer, J.W., 1971. Fourier series representation of the position of the Sun. Search 2, 172–173.

Swift, L.W., 1976. Algorithm for solar radiation on mountain slopes. Water Resour. Res. 12, 108–112.

The Astronomical Almanac, 1985, 1986 ed. US Gov. Printing Office, Washington, DC.

Walraven, R., 1978. Calculating the position of the Sun. Solar Energy 20, 393–397.

Walraven, R., 1979. Erratum. Solar Energy 22, 195.

Wilkinson, B.J., 1981. An improved FORTRAN program for the rapid calculation of the solar position. Solar Energy 27, 67–68.